



MODELO DE COMPORTAMIENTO

Diseño de Objetos Asignando
Responsabilidades

Diseño de Sistemas de Información
Universidad Tecnológica Nacional – Facultad Regional Mendoza

Marcos Espeche

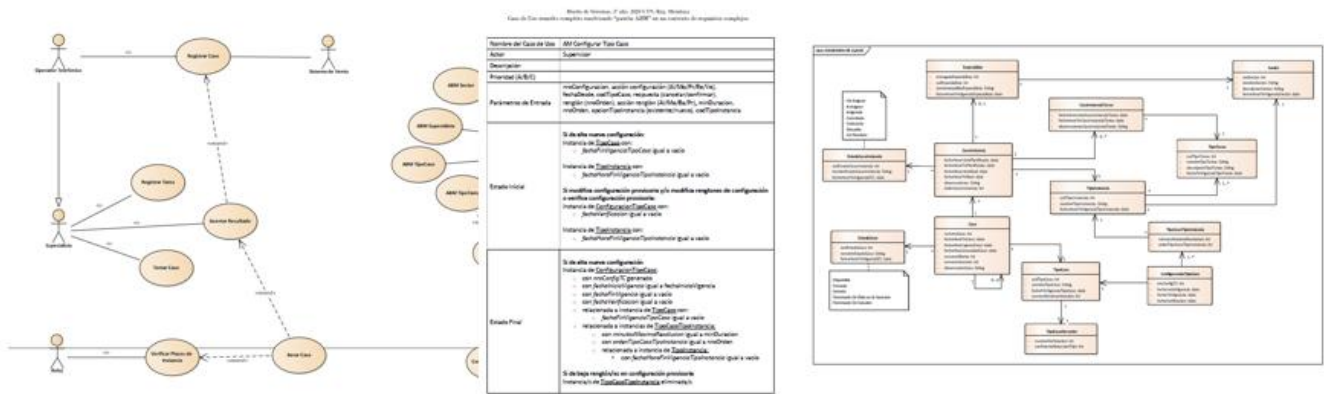
Contenido

Introducción	2
Responsabilidades	2
Patrones de Diseño GRASP	6
Cohesión y Acoplamiento	6
Alta Cohesión.....	6
Bajo Acoplamiento.....	8
Relación entre Cohesión y Acoplamiento.....	10
Experto en Información.....	11
Fabricación Pura.....	14
Indirección	15
Variaciones Protegidas.....	17
Controlador	20
DTO	23
Conclusión	28
Preguntas de autoevaluación	28
Bibliografía	29

Introducción

Se recomienda, antes de iniciar con este apunte, haber repasado los conceptos vistos en la clase de Ingeniería Directa.

Para elaborar el modelo de comportamiento es necesario tener como punto de partida distintos artefactos, que pueden tener un mayor o menor grado de completitud: principalmente modelo de casos de uso, diagrama de clases de entidad, y descripción del flujo de sucesos de casos de uso. Para realizarlos, hubo que pensar en qué datos necesita persistir el sistema, qué funcionalidades ofrece, prototipos de pantallas, y un pseudocódigo de cómo se llevarán a cabo los distintos casos de uso.



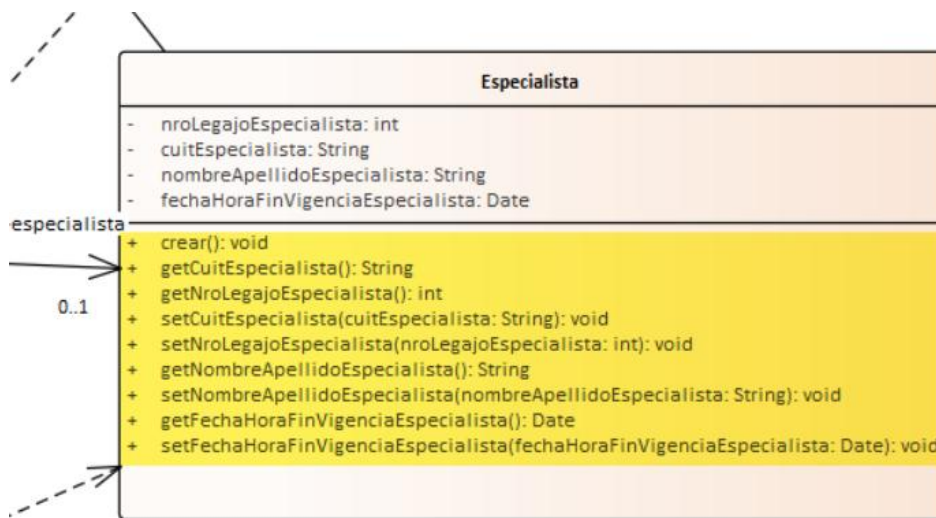
En diseño, a diferencia del análisis, se baja el nivel de abstracción a un nivel más cercano a la implementación. Sin embargo, las clases que se han modelado no alcanzan para que el sistema sea funcional, ya que solo *conocen* datos. Hasta ahora no se había pensado en cuáles van a ser las clases que *hagan* cosas.

En este eje temático se estudiará cómo describir la **realización de los casos de uso**, es decir, la forma en la que van a colaborar los distintos **objetos de diseño** para llevar a cabo los casos de uso, utilizando algunos patrones y principios de diseño. Dedicar esfuerzo a este modelo no solo conduce a tener una perspectiva más detallada de cómo se comporta el sistema internamente, sino también permite refinar y completar los modelos anteriores.

Responsabilidades

Para poder describir la realización de los casos de uso es necesario asignar **responsabilidades**. Las responsabilidades son contratos u obligaciones de un clasificador [1]. Dichos clasificadores se pueden entender como objetos, componentes, subsistemas, sistemas, o cualquier otro nivel de granularidad.

En la materia, se van a interpretar como las obligaciones de un objeto, que se llevan a cabo mediante los métodos que tiene. La relación entre responsabilidad y método no es 1:1, sino 1:N.

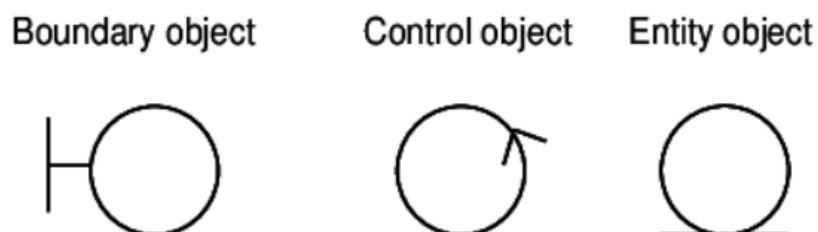


Se podría clasificar a las responsabilidades en dos tipos:

- Responsabilidades de *conocer*
 - Conocer los datos privados encapsulados
 - Conocer los objetos relacionados
 - Conocer datos que se pueden derivar o calcular
- Responsabilidades de *hacer*
 - Hacer algo en sí mismo, como crear un objeto o realizar algún cálculo
 - Iniciar una acción en otros objetos
 - Controlar y coordinar acciones en otros objetos

Asignar responsabilidades no es una tarea trivial. Además de que el sistema cumpla con los requerimientos relevados, se debe cumplir el principal objetivo del diseño de software: las responsabilidades se asignan de forma tal que el sistema sea **fácil de mantener**. La mantenibilidad es uno de los principales atributos que conforman la calidad del diseño y se define como *la facilidad con la que un sistema puede ser modificado, adaptado, corregido, o mejorado sin introducir nuevos errores*.

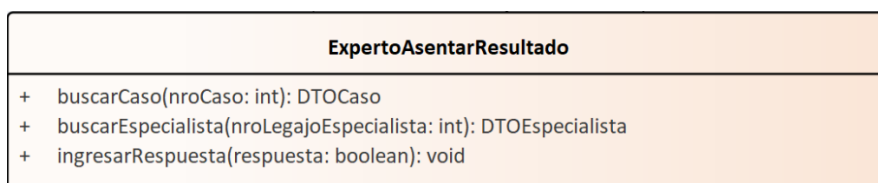
Un buen primer criterio a la hora de asignar responsabilidades es basarse en los estereotipos del análisis, descriptos en el Proceso Unificado [2]. Cada estereotipo maneja un tipo de responsabilidad distinta:



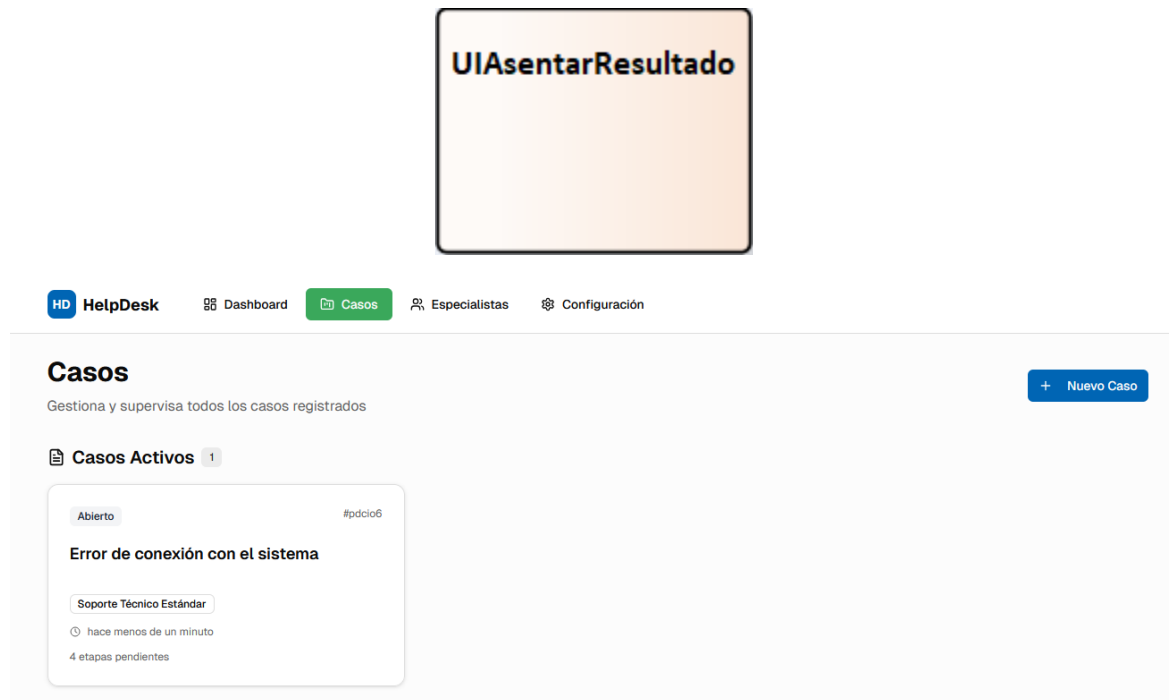
- Las **clases de entidad** son aquellas que *conocen* datos y que a menudo son persistentes. Se corresponden con las clases del diagrama de clases de entidad.



- Las **clases de control** poseen lógica, es decir, responsabilidades *de hacer*. Por ejemplo, aquellas clases con la lógica de negocio de los casos de uso, conexión con servicios externos, cálculos, acceso a la base de datos, etc. Como mínimo, cada caso de uso debe tener una clase de este estereotipo. Hay distintas formas de organizarlas, ya sea una por caso de uso o una por concepto de dominio. En la materia se va a optar por el primer enfoque, donde la clase que conoce la lógica de un caso de uso se denomina “Experto” seguido del nombre del caso de uso (puesto que es quien conoce cómo llevarlo a cabo).

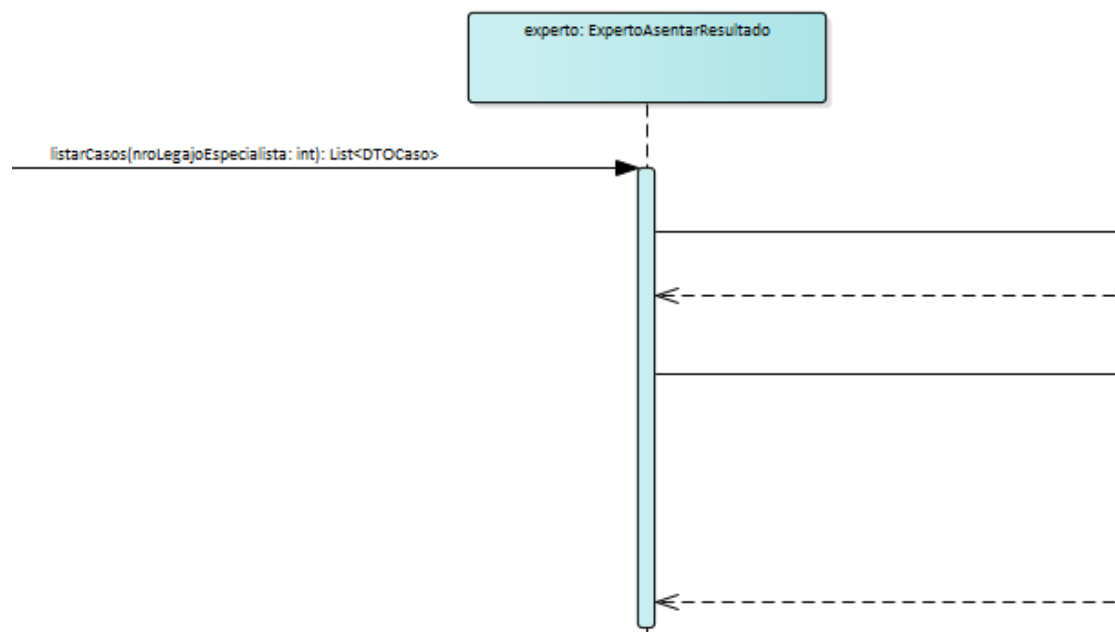


- Las **clases de interfaz** son las que permitirán la interacción con los actores de los casos de uso. Consisten en ventanas, *labels*, botones, formularios, y cualquier otro componente gráfico. En la materia se utilizará una única clase como abstracción de la pantalla correspondiente a un caso de uso. En la práctica, podría consistir en N clases (como en Java Swing, donde cada tipo de componente se corresponde con una clase), o podría no tener ninguna clase asociada (por ejemplo, si el sistema tiene una interfaz web podría no utilizarse lenguajes orientados a objetos sino HTML)



Es necesario un artefacto para poder visualizar las elecciones en la asignación de responsabilidades. Para este propósito, en la cátedra se utilizará un **diagrama de secuencia**, que forma parte de los diagramas de interacción.

Por ejemplo, en la siguiente porción de diagrama de secuencia se puede ver que se le asignó a la clase *ExpertoAsentarResultado* la responsabilidad de listar Casos a los Especialistas, debido a que tiene el método *listarCasos*. Ese método a su vez llama a otros para poder realizar su funcionalidad.



Patrones de Diseño GRASP

Sin embargo, no alcanza únicamente con los estereotipos del análisis. Incluso tomándolos como base, se puede llegar a una baja mantenibilidad. A lo largo de la historia del diseño de software se ha encontrado que hay problemas recurrentes en la mayoría de sistemas. Dichos problemas, si son de la misma naturaleza, pueden ser solucionados de la misma forma.

Según Larman [1], un **patrón** se define como un par problema-solución con nombre que se puede aplicar en nuevos contextos. Son soluciones testeadas y aplicables en escenarios comunes que ayudan a llegar a diseños mantenibles y escalables.

Los Patrones GRASP (*General Responsibility Assignment Software Patterns*) describen los principios fundamentales del diseño orientado a objetos y la asignación de responsabilidades, expresados como patrones.

Cohesión y Acoplamiento

Alta Cohesión

Problema	¿Cómo mantener la complejidad manejable?
Solución	Asignar responsabilidades de manera que la cohesión permanezca alta.

La **cohesión** es una medida del grado de focalización de las responsabilidades de un elemento (entendiéndose por elemento a una clase, un componente, un subsistema, etc.). Una clase con una alta cohesión tiene responsabilidades relacionadas, y no hace una gran cantidad de trabajo. Una clase con una baja cohesión hace muchas cosas no relacionadas, o hace demasiado trabajo.

Las clases con una baja cohesión son extremadamente complejas, ya que son:

- Dificiles de entender: Demasiadas responsabilidades no relacionadas hace que se desvirtúe el propósito de la clase, reduciendo su legibilidad. Se gasta mucho tiempo en entender la clase.
- Dificiles de reutilizar: Algunas responsabilidades pueden ser requeridas en diversos puntos del sistema. Se nos obligaría a arrastrar todas las responsabilidades de la clase (junto con todas sus dependencias) sin que sea intuitivo cuáles serán utilizadas, o a duplicar código.
- Dificiles de mantener: Al no estar focalizadas las responsabilidades, no es intuitivo identificar cuáles clases deben ser modificadas al hacer mantenimiento del sistema.
- Dificiles de rastrear errores: Si una clase tiene demasiadas responsabilidades, tiene muchos motivos para ser modificada. Si en una de esas modificaciones se introdujo un error, es difícil averiguar en cuál de todas ellas fue.

- Poco testeables: Si una clase realiza demasiadas responsabilidades, y además están acopladas entre sí, el testing se dificulta. Esto es porque en tests de clase no se pueden utilizar *mocks/stubs* para los métodos de la misma, entonces resulta complicado aislar y focalizar las pruebas.

Por ejemplo, se puede pensar en la siguiente clase:

```
public class Usuario {  
  
    private String nombre;  
  
    private String email;  
  
    public void guardarUsuario() { ... }  
  
    public void enviarEmail(String destinatario, String mensaje) { ... }  
  
    public void generarReportePDF() { ... }  
  
    public void procesarPago(double monto) { ... }  
  
    public boolean validarTarjeta(String numero) { ... }  
  
}
```

- Difícil de entender: ¿Por qué una clase de entidad posee lógica de negocio? Un desarrollador que tenga que realizar mantenimiento de esta clase deberá gastar mucho tiempo leyendo el código entero para entender qué es lo que hace, corriendo riesgo de introducir nuevos errores.
- Difícil de reutilizar: Si el envío de mails es requerido en otras partes del sistema, hay que copiar y pegar ese fragmento de código o arrastrar la clase *Usuario* entera (junto con todas sus dependencias) al resto del sistema, sin que quede claro por cuál de todas sus responsabilidades la necesitan. Además, ese método puede no ser genérico y servir únicamente dentro de la clase *Usuario*.
- Difícil de mantener: Si se solicita realizar cambios en el envío de mails o en el procesamiento de pagos, para un desarrollador es poco intuitivo pensar que tiene que ir a la clase *Usuario* para ello. Podría ser una funcionalidad que está dispersa en distintas clases que no parecieran estar relacionadas para ese propósito.
- Difícil de rastrear errores: Distintos desarrolladores modifican la clase *Usuario*, cada uno de ellos por motivos distintos. Si en un entorno de producción aparece un error no va a ser fácil identificar el cambio de quién fue el que causó el problema.
- Poco testeable: Si el método *procesarPago* llama a *validarTarjeta*, *generarReportePDF*, y *enviarEmail*, no se puede testear su funcionalidad de forma aislada sin hacer una prueba de integración con el resto de métodos.

Bajo Acoplamiento

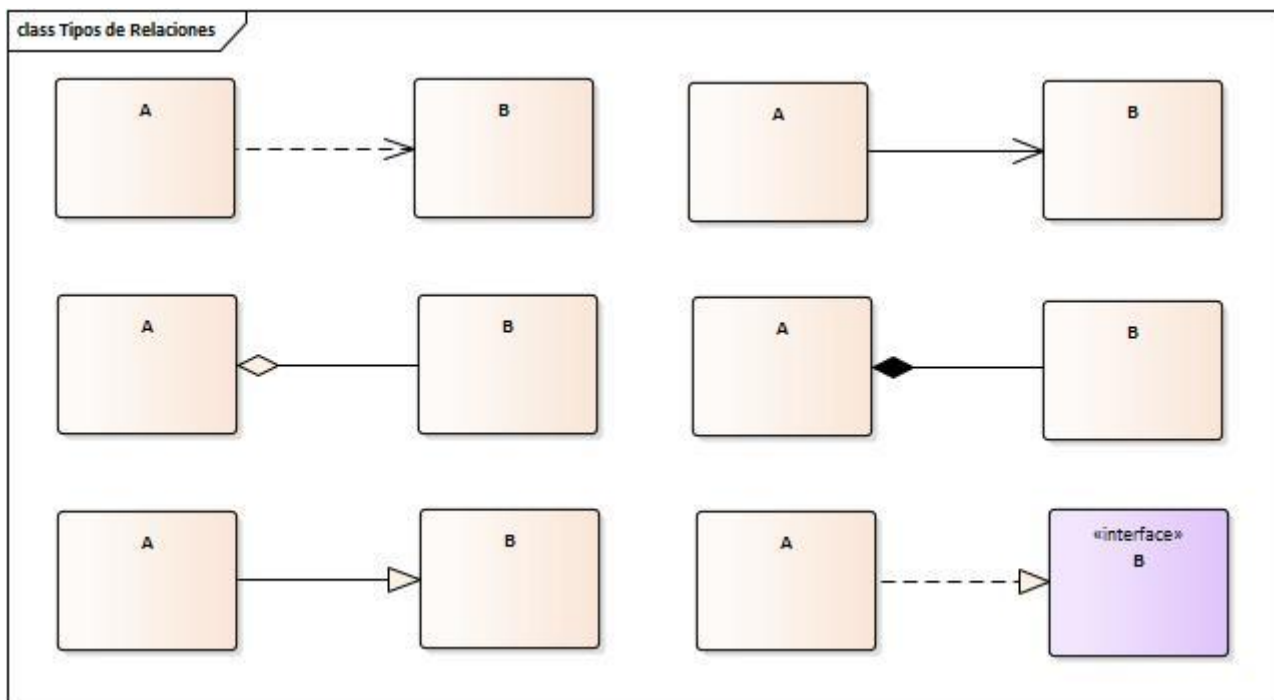
Problema	¿Cómo soportar bajas dependencias, bajo impacto al cambio, e incremento de la reutilización?
Solución	Asignar responsabilidades de manera que el acoplamiento permanezca bajo.

El **acoplamiento** es una medida de la fuerza con que un elemento está conectado a, tiene conocimiento de, confía en, otros elementos (entendiéndose por elemento a una clase, un módulo, componente, subsistema, sistema, etc.). Un elemento con bajo (o débil) acoplamiento no depende de demasiados otros elementos. Un elemento con alto (o fuerte) acoplamiento confía en muchos otros elementos.

Las clases con un alto acoplamiento traen los siguientes problemas:

- Propagación de modificaciones: Con un alto acoplamiento, las clases son muy dependientes entre sí. Si hay un cambio en una de las dependencias de una clase, es posible que ésta deba ser modificada también para poder seguir funcionando luego de la modificación.
- Difícil de entender de forma aislada: Para entender qué hace una clase, es necesario entender todas las clases de las que depende y cómo interactúan. No se puede comprender una pieza del sistema sin el esfuerzo de recordar y entender todas sus dependencias.
- Baja reutilización: Si una clase con un alto acoplamiento es requerida en distintos puntos del sistema, hay que arrastrar también todas sus dependencias.
- Baja testeabilidad: Para poder focalizar las pruebas correctamente hay que dedicar mucho esfuerzo a la configuración de *mocks/stubs* para todas las dependencias de la clase. Un cambio en alguna de las dependencias puede romper los tests.

El acoplamiento se genera siempre que haya algún grado de conocimiento sobre el funcionamiento de otro elemento. En este apunte se hará foco en el acoplamiento entre clases. Con cualquier tipo de relación entre clases se genera acoplamiento. Aunque algunas generan un acoplamiento más fuerte, como la composición o la herencia.



Sin embargo, el tipo de relación no es la única variable a la hora de evaluar el grado de acoplamiento entre clases. Dos clases pueden tener una relación de asociación, pero pueden tener un acoplamiento tanto fuerte como débil. Esto se puede dar por distintas formas de acoplamiento. Para ejemplificar, algunas de ellas son las siguientes:

- Una clase conoce el orden en el que debe llamar a los métodos de otra, siendo que podría encapsularse en un único método que coordine dichas llamadas.
- Una clase puede acoplarse a los detalles internos o a la semántica específica de las estructuras de datos que recibe o retorna de otra clase. Aunque la signature del método no cambie, cambios en cómo se estructura, organiza o interpreta esa información internamente generan un acoplamiento implícito.

En ambos casos, la relación puede ser de asociación, pero el acoplamiento es elevado debido a que se conoce el funcionamiento interno de la otra clase.

Larman [1] menciona que el acoplamiento en sí no es un problema, sino estar acoplado a elementos que no son estables en alguna dimensión, como su interfaz, implementación, o mera existencia. Como diseñadores, se puede reducir el acoplamiento en muchas áreas del sistema. Pero para emplear el tiempo de la mejor manera posible, conviene focalizar los esfuerzos en aquellas partes que es realista considerar que puedan ser modificadas. **“Elige tus batallas”**. Esto se relaciona con el patrón “Variaciones Protegidas”, que será estudiado más adelante.

Por ejemplo, se puede pensar que muchas clases tienen un alto acoplamiento con la implementación de *String*, de Java. Y es cierto, hay un fuerte acoplamiento con ella, se conoce cómo funciona y qué métodos ofrece (*.concat()*, *.join()*, *.equals()*, etc.). Pero es improbable que Java cambie la implementación de la clase

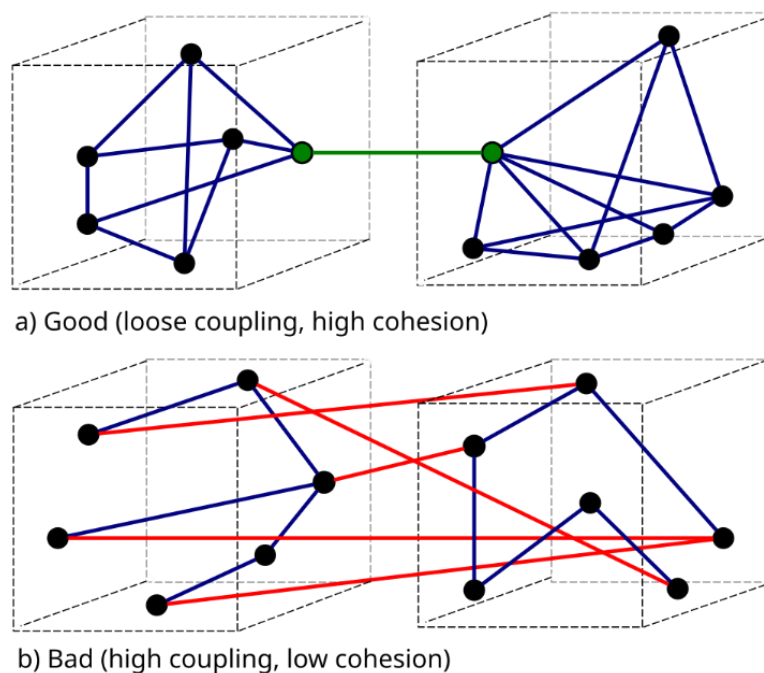
String de forma tal que tenga un impacto negativo en mantenimiento del sistema. El esfuerzo de reducir el acoplamiento con dicha clase es tan elevado que no merece la pena dedicar tiempo y energía a ello.

Relación entre Cohesión y Acoplamiento

Tanto la cohesión como el acoplamiento son **principios evaluativos**. Esto significa que no se aplican como una solución en sí misma, sino como una forma de evaluar alternativas, priorizando aquellas que favorezcan una alta cohesión y un bajo acoplamiento.

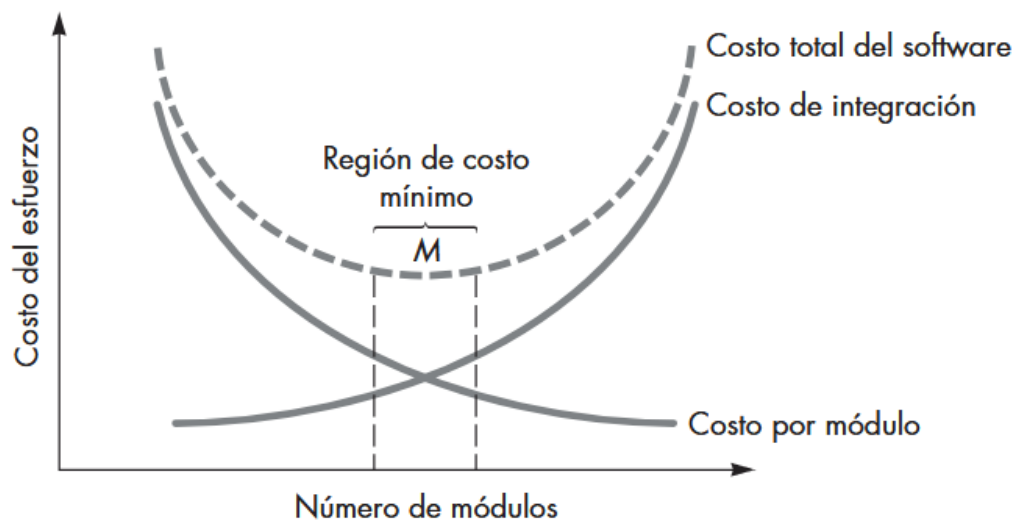
Fuertemente relacionado a la cohesión y el acoplamiento, hay otro principio de diseño de sistemas que es promover el **diseño modular**. La modularidad es la capacidad de un sistema de dividirlo en un conjunto de módulos altamente cohesivos y débilmente acoplados, de forma tal que se pueda modificar un módulo entero sin tener que hacer grandes cambios en el resto del sistema.

En la siguiente imagen, cada cubo representa un módulo. En el caso a), se ve cómo las responsabilidades de cada módulo están relacionadas (alta cohesión), y los módulos no conocen detalles de implementación del resto (bajo acoplamiento). El caso b), por el contrario, resulta en un sistema que es extremadamente complicado de mantener.



A la hora de modularizar el sistema, tomando el principio de alta cohesión, lo más cohesivo es que cada elemento realice una única responsabilidad de granularidad fina. Inevitablemente, esto conduce a tener un sistema conformado por muchos elementos que deben colaborar entre sí para llevar a cabo responsabilidades mayores, lo cual se traduce en un alto acoplamiento. Por otro lado, reducir el acoplamiento al mínimo implica que una única clase realice todas las responsabilidades, para que no deba depender de nadie. Como ya se estudió, esto representa una solución con una muy baja cohesión.

Pressman [3] ilustra muy bien esta idea con el siguiente gráfico. El acoplamiento y la cohesión no pueden ser considerados de forma aislada ya que se afectan mutuamente. Se debe optar por una solución que tenga un acoplamiento que sea aceptablemente bajo, y una cohesión aceptablemente alta.

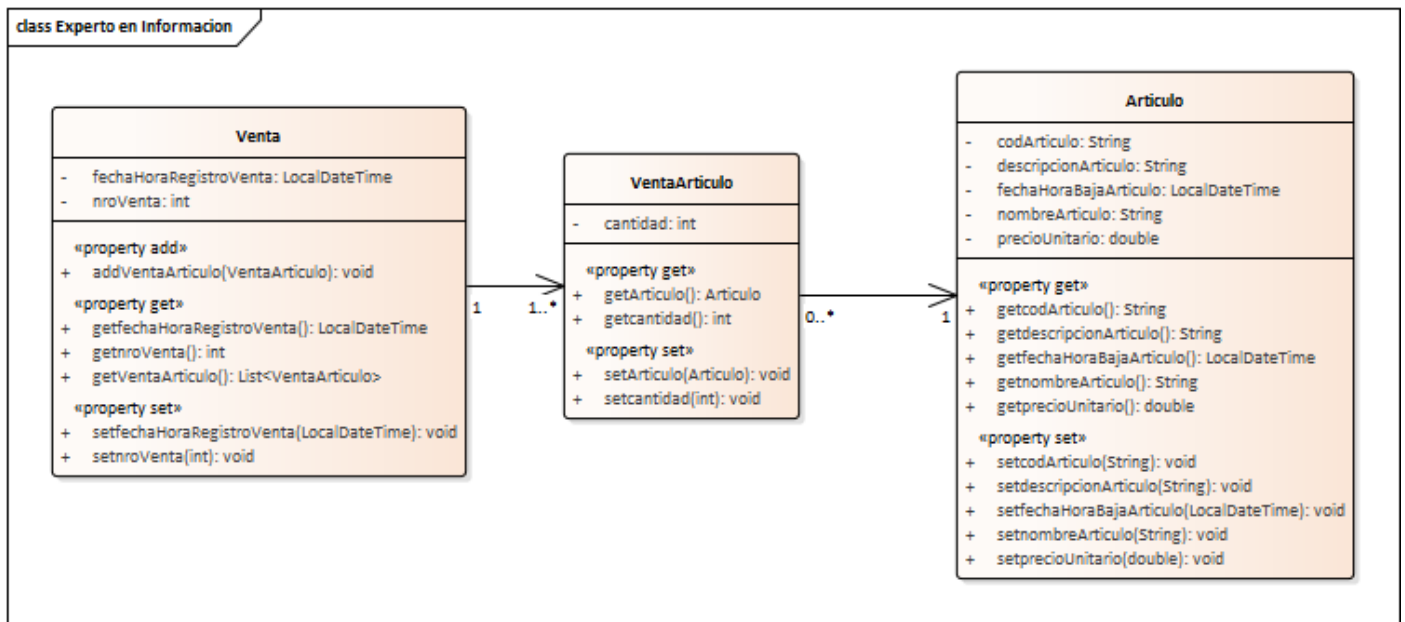


Experto en Información

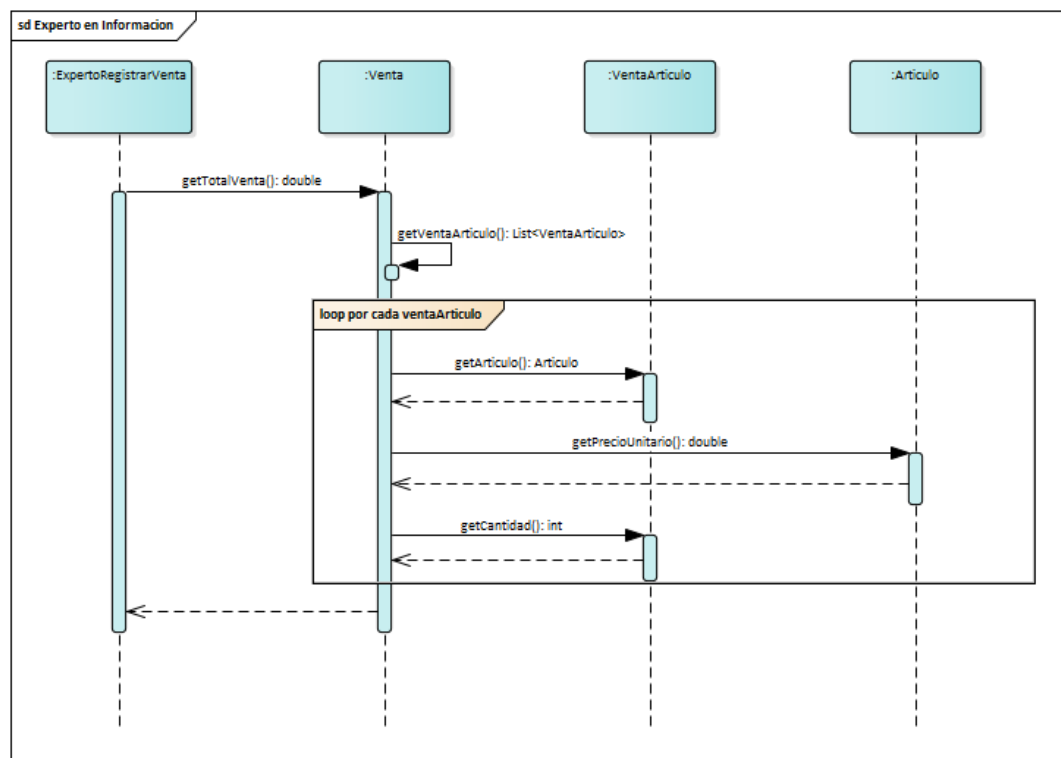
Problema	¿Cuál es un principio general para asignar responsabilidades a los objetos?
Solución	Asignar una responsabilidad al experto en información.

Cuando se menciona que se debe asignar responsabilidades al experto en información se refiere a darle la responsabilidad a quien tenga la información necesaria para llevarla a cabo. Este patrón da una buena primera aproximación de dónde empezar a asignar responsabilidades (luego de haber utilizado los estereotipos del análisis). Busca ser una idea intuitiva, puesto que lo lógico es pensar que los objetos hacen las cosas relacionadas con la información que tienen, lo que conduciría a soluciones que tengan una alta cohesión y un bajo acoplamiento.

La asignación de una responsabilidad a un experto en información puede requerir que, para poder llevarla a cabo, deba interactuar con expertos en información “parciales”. Por ejemplo:

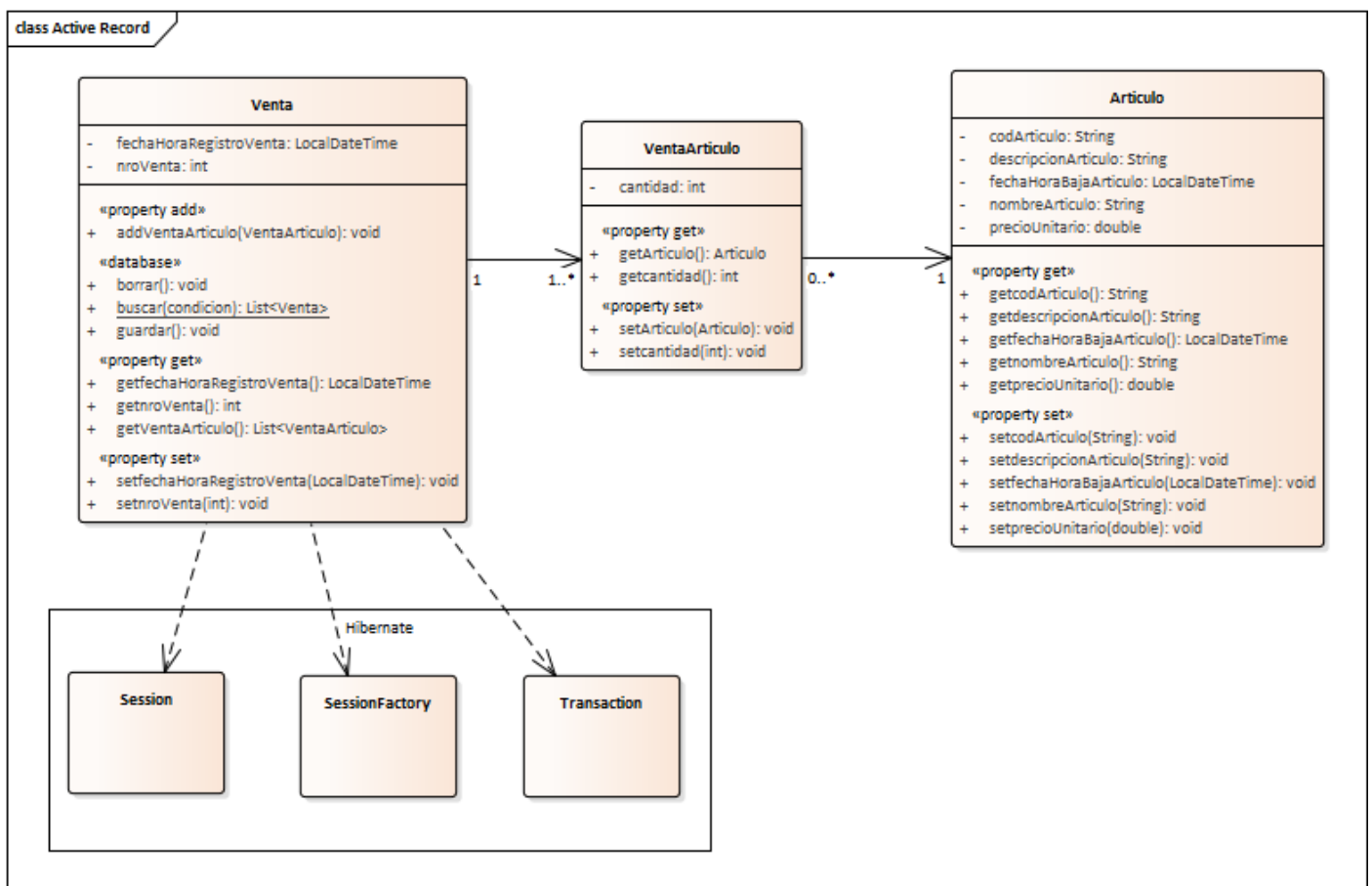


Dado el siguiente modelado, ¿a quién se le debe dar la responsabilidad de conocer el total de una venta? El patrón Experto en Información diría que debe ser quien conozca la información necesaria. Siendo que el total de una venta se compone de la sumatoria de sus subtotales, la única clase que tiene acceso a esa información es la clase *Venta*. Pero, para conocer los subtotales, cada instancia de *VentaArtículo* tuvo que consultar el precio unitario en la clase *Artículo*, haciendo que sea un Experto en Información “parcial” para la responsabilidad de conocer el total de una venta.



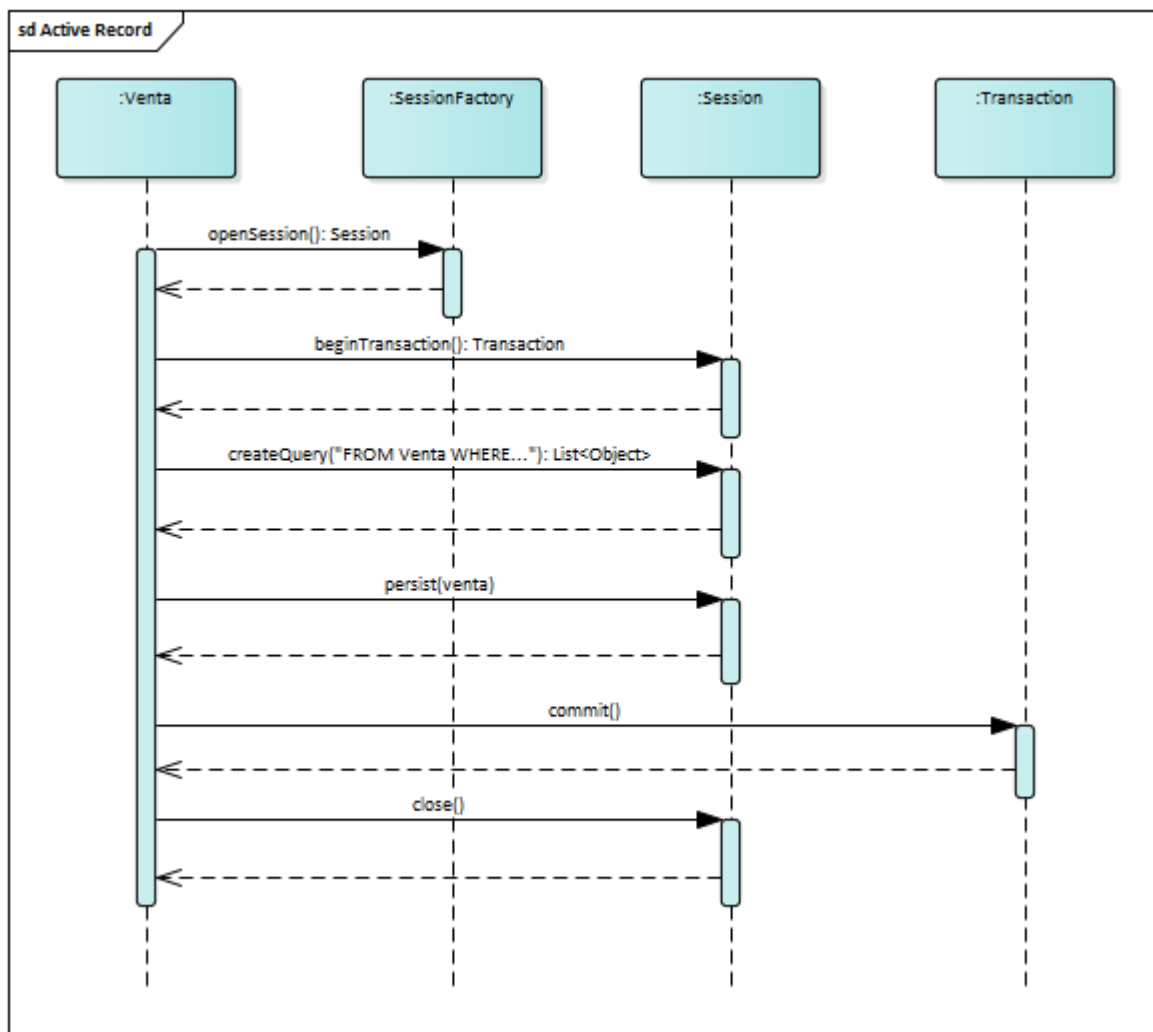
Este patrón, sin embargo, no siempre conduce a diseños deseables. Como se mencionó anteriormente, se debe evaluar la cohesión y el acoplamiento en cada una de las soluciones candidatas. Por ejemplo, para la responsabilidad de persistir en una base de datos relacional una venta, el patrón Experto en Información diría que la clase *Venta* es una buena candidata para llevarla a cabo, puesto que conoce todos los datos que se requieren persistir. Análogamente, cada entidad sería responsable de persistirse a sí misma.

En la siguiente imagen se ve que la entidad *Venta* (luego, cada entidad) tiene dependencias con las clases de Hibernate, ya que son utilizadas para interactuar con la base de datos.



Esto trae consigo tres problemas:

- Una baja cohesión, puesto que la responsabilidad de persistir un objeto en la base de datos no se relaciona con la responsabilidad de “ser una venta”.
- Un alto acoplamiento con el mecanismo de persistencia, ya sea con Hibernate, JPA, JDBC, o cualquier otro que se desee implementar. Las entidades deberán conocer cómo funcionan dichos frameworks/librerías y estarán acopladas a un conjunto de clases relacionadas a la persistencia (en este caso, simplificadas usando un paquete de UML).
- Duplicidad de código y responsabilidades, ya que cada entidad deberá tener lógica similar para interactuar con la base de datos. Dicha lógica corresponde a responsabilidades que debieran estar agrupadas en otro lugar.



La imagen anterior muestra una simplificación de cómo la clase *Venta* interactuaría con Hibernate para persistir y realizar consultas a la base de datos. Lo mismo ocurriría para el resto de entidades del sistema, como *Artículo* y *VentaArtículo*.

Si recordamos que el primer principio que se debe utilizar para la separación de responsabilidades es utilizar los estereotipos del análisis, se podría considerar trasladar la responsabilidad a las clases de control de cada caso de uso (por ejemplo, *ExpertoAsentarResultado*), puesto que una entidad no debería tener lógica. Sin embargo, nos encontraríamos con el mismo problema puesto que cada clase de control debería conocer cómo interactuar con Hibernate, reduciendo así su cohesión y generando un fuerte acoplamiento con el framework. Para resolver esto, necesitamos recurrir a otros patrones.

Fabricación Pura

Problema	¿Qué objetos deberían tener la responsabilidad cuando no quiere violar los objetivos de Alta Cohesión y Bajo Acoplamiento, u otros, pero las soluciones que ofrecen otros patrones (Experto en Información, Creador, etc.) no son adecuadas?
----------	--

Solución	Asigne un conjunto de responsabilidades altamente cohesivo a una clase artificial o de conveniencia que no representa un concepto del dominio del problema
----------	--

Un objeto **Fabricación Pura** es una clase “inventada”, una “fabricación de la imaginación”. Por lo tanto, no existía previamente en el dominio del sistema, las responsabilidades asignadas deberían ser altamente cohesivas y con un bajo acoplamiento, de manera que resulte en un diseño limpio o “puro”.

Utilizar el patrón Fabricación Pura soporta la Alta Cohesión, ya que las responsabilidades se factorizan en una clase de granularidad fina que se centra en un conjunto muy específico de tareas relacionadas. Además, aumenta el potencial de reutilización, ya que las responsabilidades asignadas a una fabricación pura podrían tener aplicación en distintas áreas del sistema.

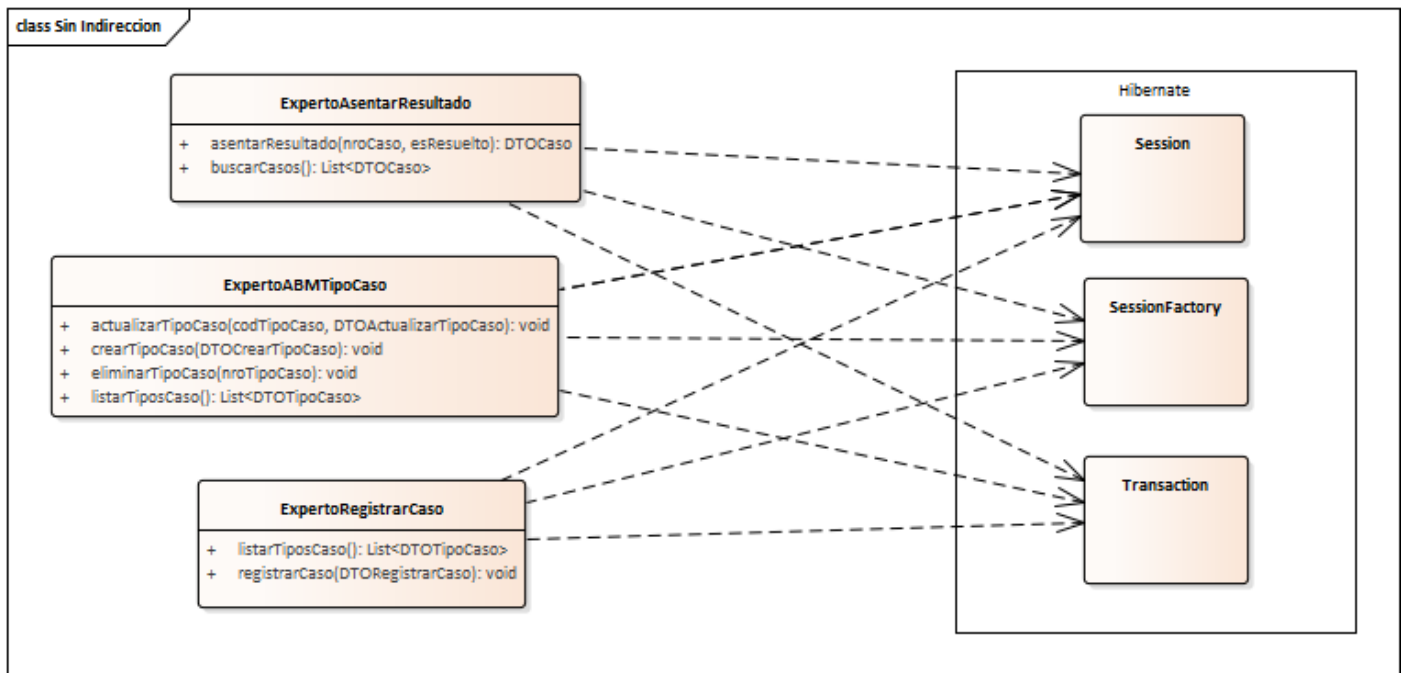
Este patrón es utilizado no solo en sí mismo, sino que otros patrones GRASP (y patrones GoF que se estudiarán más adelante) son fabricaciones puras. Para resolver el problema de la persistencia expuesto en la sección anterior se puede utilizar este patrón, como se verá a continuación.

Indirección

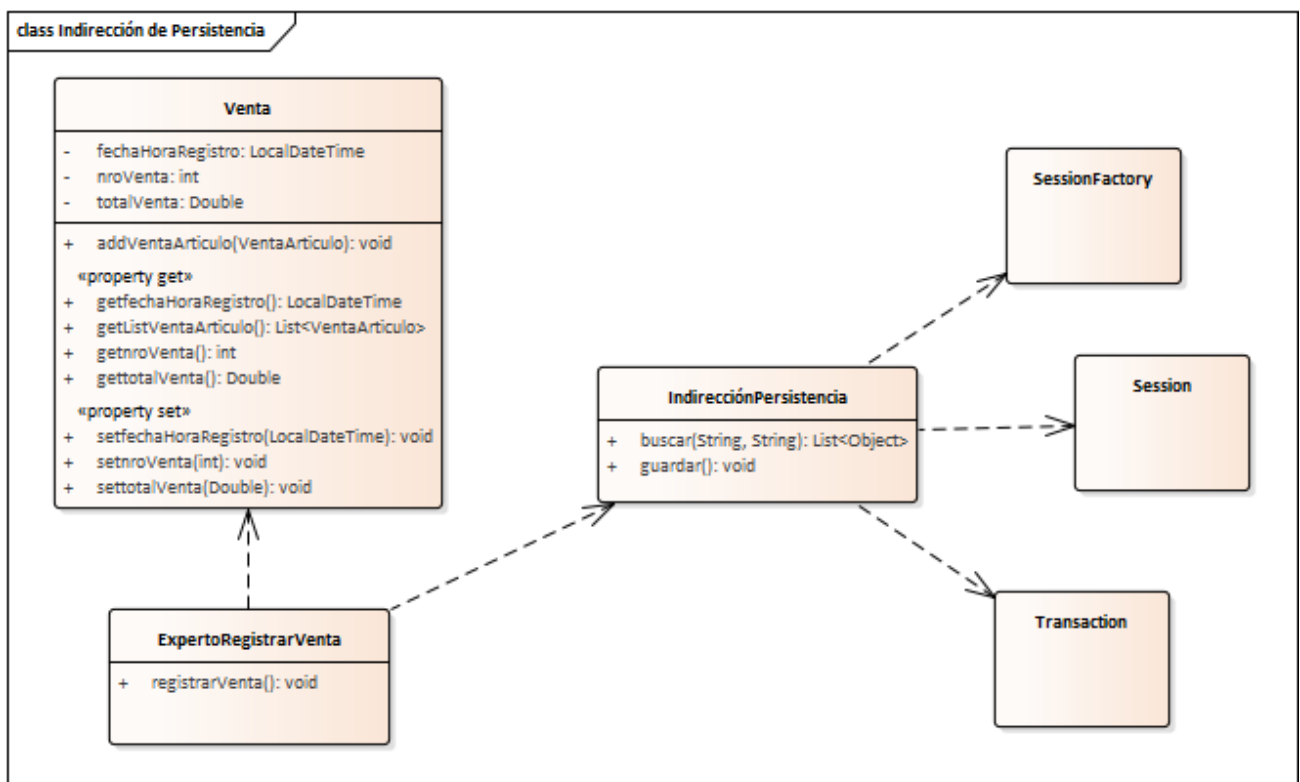
Problema	¿Dónde asignar una responsabilidad, para evitar el acoplamiento directo entre dos o más cosas? ¿Cómo desacoplar los objetos de manera que se soporte el bajo acoplamiento y el potencial para reutilizar permanezca más alto?
Solución	Asigne la responsabilidad a un objeto intermedio que medie entre otros componentes o servicios de manera que no se acoplen directamente.

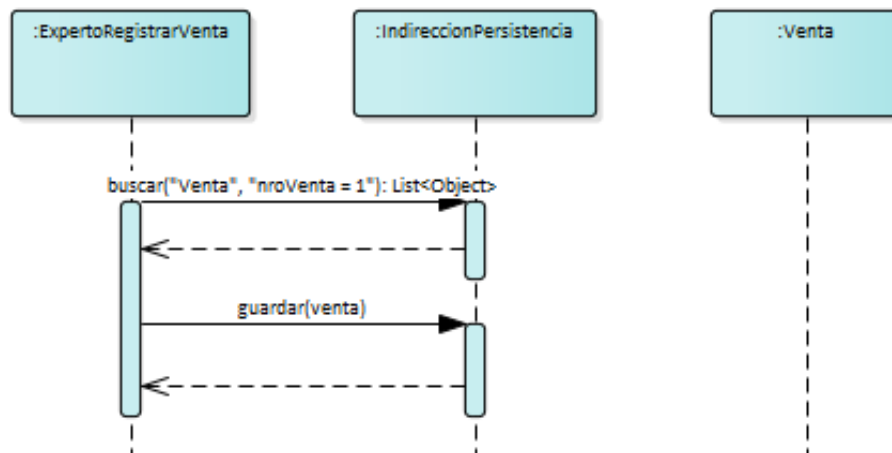
El patrón **Indirección** es una aplicación directa de Fabricación Pura. Es un caso particular donde se crea un nuevo elemento con el propósito de reducir el acoplamiento de una clase con otras.

Retomando el caso de la responsabilidad de la persistencia de objetos, se tenía el problema de que la clase *Venta* conoce cómo persistirse, generando en una baja cohesión, alto acoplamiento (en este caso, con el conjunto de clases de Hibernate), y baja reutilización. Al trasladar el problema a las clases de control de los casos de uso el problema persiste, puesto que conocer el detalle de la interacción con la base de datos no está relacionado a la lógica de un caso de uso (ya que es un aspecto no funcional), disminuyendo la cohesión de dichas clases. Por otro lado, se genera un alto acoplamiento y una baja reutilización puesto que realizar un cambio en el mecanismo de persistencia implica realizar modificaciones en muchas clases del sistema.



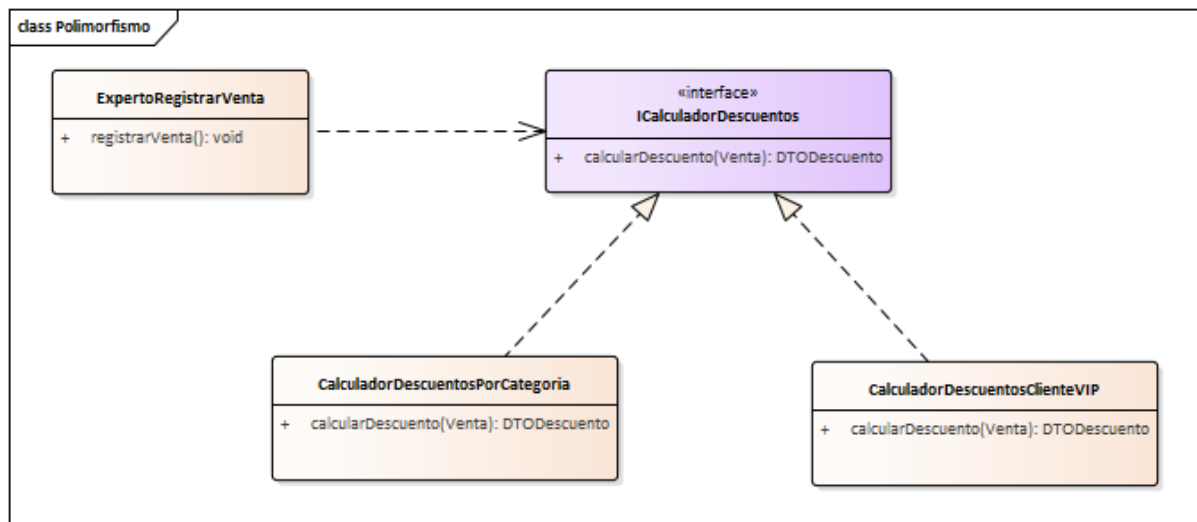
Se puede utilizar una fabricación pura que actúe como Indirección, para quitarle a las entidades la responsabilidad de persistirse a sí mismas, y quitarles a las clases de control la responsabilidad de conocer lógica no relacionada a los casos de uso. Esto desacopla a las clases involucradas de la interacción con Hibernate y además favorece una alta reutilización, ya que cualquier clase que requiera interactuar con la base de datos tiene que pasar por la indirección de persistencia, que encapsula y abstrae esa complejidad de manera centralizada.





Para estudiar este tema con más profundidad, se puede consultar el apunte de cátedra “Framework de Persistencia”, que desarrolla cómo diseñar la indirección de persistencia utilizando el patrón Fachada, Decorador, y DTO, y el apunte de cátedra “Búsqueda con Indirección”.

Otra forma de implementar el patrón Indirección es mediante una abstracción (como una interfaz o una clase abstracta) y el uso de Polimorfismo (otro patrón GRASP). En este caso, se busca desacoplar a una clase de posibles implementaciones que resuelven una misma funcionalidad.



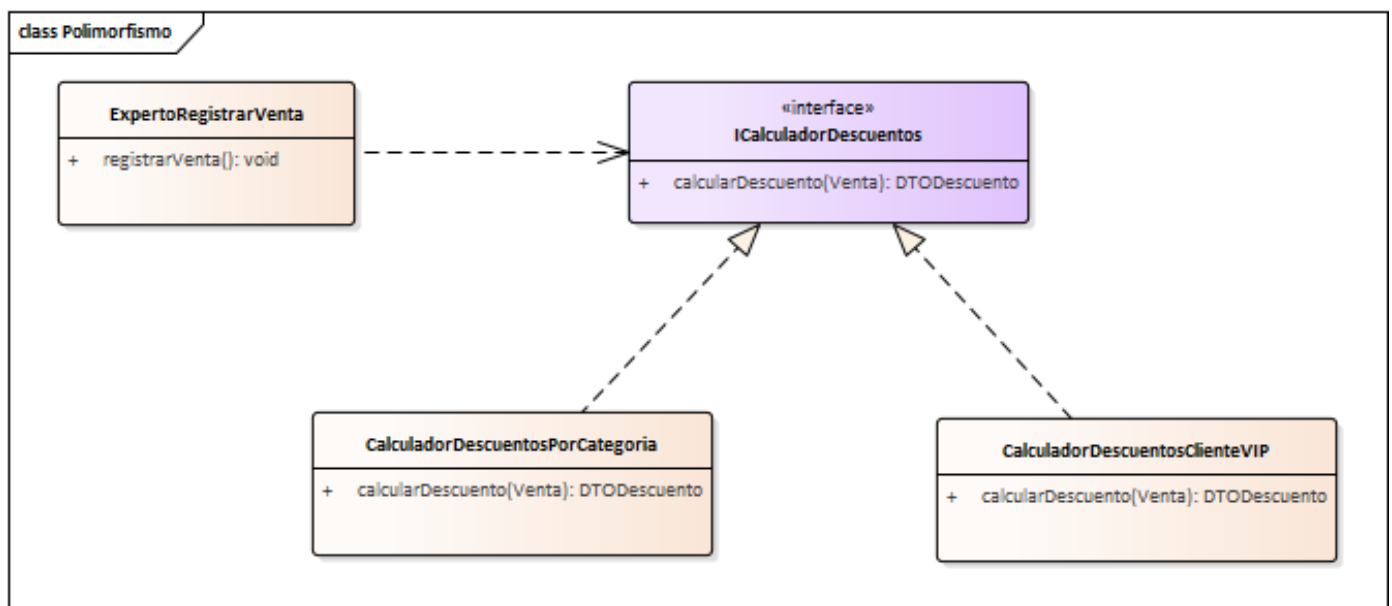
Variaciones Protegidas

Problema	¿Cómo diseñar objetos, subsistemas y sistemas de manera que las variaciones o inestabilidades en estos elementos no tengan un impacto no deseable en otros elementos?
Solución	Identifique los puntos de variaciones previstas o de inestabilidad; asigne responsabilidades para crear una interfaz estable alrededor de ellos.

El término “**interfaz**” tiene muchas acepciones en el ámbito de la ingeniería en sistemas. En este caso, “interfaz” se refiere a un punto de acceso a un objeto, componente, módulo, subsistema, o sistema. Se verá que hay muchas formas de implementarlo. La palabra “estable” se refiere a que soporte los cambios en el tiempo.

Se lo considera equivalente a los principios de **ocultación de la información** (ocultar información sobre decisiones de diseño a otros módulos) y al **principio de abierto-cerrado** (los módulos deberían estar abiertos a la extensión, pero cerrados a la modificaciones que afecten a sus clientes).

Una forma de aplicar Variaciones Protegidas es mediante el uso de interfaces (clases abstractas puras) y Polimorfismo, como se vio en el uso del patrón Indirección. En este caso, el punto de variación es el cálculo de descuentos. Se utiliza una *interfaz estable*, al exponer únicamente una signatura de un método en lugar de los posibles detalles de implementación.



Sin embargo, no es la única forma de protegerse de variaciones. Este principio se utiliza de formas muy diversas. Esta es una lista no exhaustiva de otras formas de aplicar variaciones protegidas:

- Encapsulamiento: El principio de encapsulamiento de la orientación a objetos es una forma de ocultar decisiones de diseño sobre las clases, mediante modificadores de acceso (public, private, protected, etc.)
- Ley de Demeter: Una forma de aplicar encapsulamiento, también llamada “No hables con extraños”. Busca proteger a las clases de la estructura interna de las entidades, cuando hay que recorrer una gran estructura de datos para llegar al dato necesario. En el siguiente ejemplo, un cambio en las relaciones entre las entidades podría afectar a ciertas partes del sistema, al tener que modificar la forma en la que se navegan los datos. En lugar de eso, la entidad *Venta* debería contener un punto de acceso simplificado para obtener el titular de la cuenta, para que de forma centralizada se pueda navegar a ese dato sin que el resto del sistema conozca cómo se estructuran las entidades.

```
public void metodoFragil() {  
    TitularCuenta titular = venta.getPago().getCuenta().getTitularCuenta();  
    //...  
}  
public void metodoMejorado() {  
    TitularCuenta titular = venta.getTitularCuenta();  
    // getTitularCuenta encapsula la navegación desde  
    Venta -> Pago -> Cuenta -> TitularCuenta,  
    de forma que es transparente la estructura de las entidades para el resto  
    del sistema  
}
```

- Virtualización: El uso de máquinas virtuales o contenedores (como Docker) permite proteger a las aplicaciones de variaciones en el entorno de ejecución mediante interfaces estables, como la interfaz de una máquina virtual o las interfaces estándar del sistema operativo, que ocultan los detalles de la infraestructura real, versiones de dependencias, etc.
- Correcta definición de signatura de métodos: Una clase que tenga una signatura de métodos genérica y robusta protege de los cambios internos que pueda tener dicha clase. Siempre que la signatura y el significado de los parámetros y retorno se mantenga igual, se protege de las inestabilidades.

Según Larman, se podrían clasificar los puntos de cambio en dos tipos [1]:

- Puntos de variación: variaciones en el sistema actual, existentes en los requisitos actuales. Se tiene la certeza de la variación.
- Puntos de evolución: puntos especulativos de variación que podrían aparecer en el futuro, pero en los requisitos actuales no están presentes.

El principio de Variaciones Protegidas se aplica tanto a los puntos de variación como a los puntos de evolución. Es importante considerar que el coste de “futuras necesidades” especulativas en los puntos de evolución pueden pesar más que un diseño simple. A veces es más eficiente refactorizar el diseño cuando se tenga la certeza del cambio antes que pensar para la flexibilidad extrema. La experiencia del diseñador es un factor importante a considerar cuando se plantea la probabilidad y el impacto de cambios no anticipados. Aplicar Variaciones Protegidas de forma indiscriminada ralentiza el diseño y desarrollo. Bien utilizado, facilita el mantenimiento en el largo plazo. Nuevamente, **elige tus batallas**.

Para ejemplificar una aplicación de variaciones protegidas: Una de las decisiones de diseño que se ha tomado en este apunte, y que en ciertos proyectos se puede considerar un punto de variación, es el mecanismo de persistencia. Se ha utilizado Hibernate, pero podría requerirse migrar a JPA o incluso alguna librería para bases de datos no relacionales como MongoDB. En estos casos, hay que proteger de esta variación al resto de clases que utilicen la Indirección de Persistencia: ¿de qué formas se puede ocultar las decisiones de diseño internas de dicha clase? Este tema es desarrollado con mayor profundidad en el apunte “Framework de Persistencia”, aunque también será trabajado y aplicado a lo largo del cursado en los diseños realizados, no solo para la indirección de persistencia.

Controlador

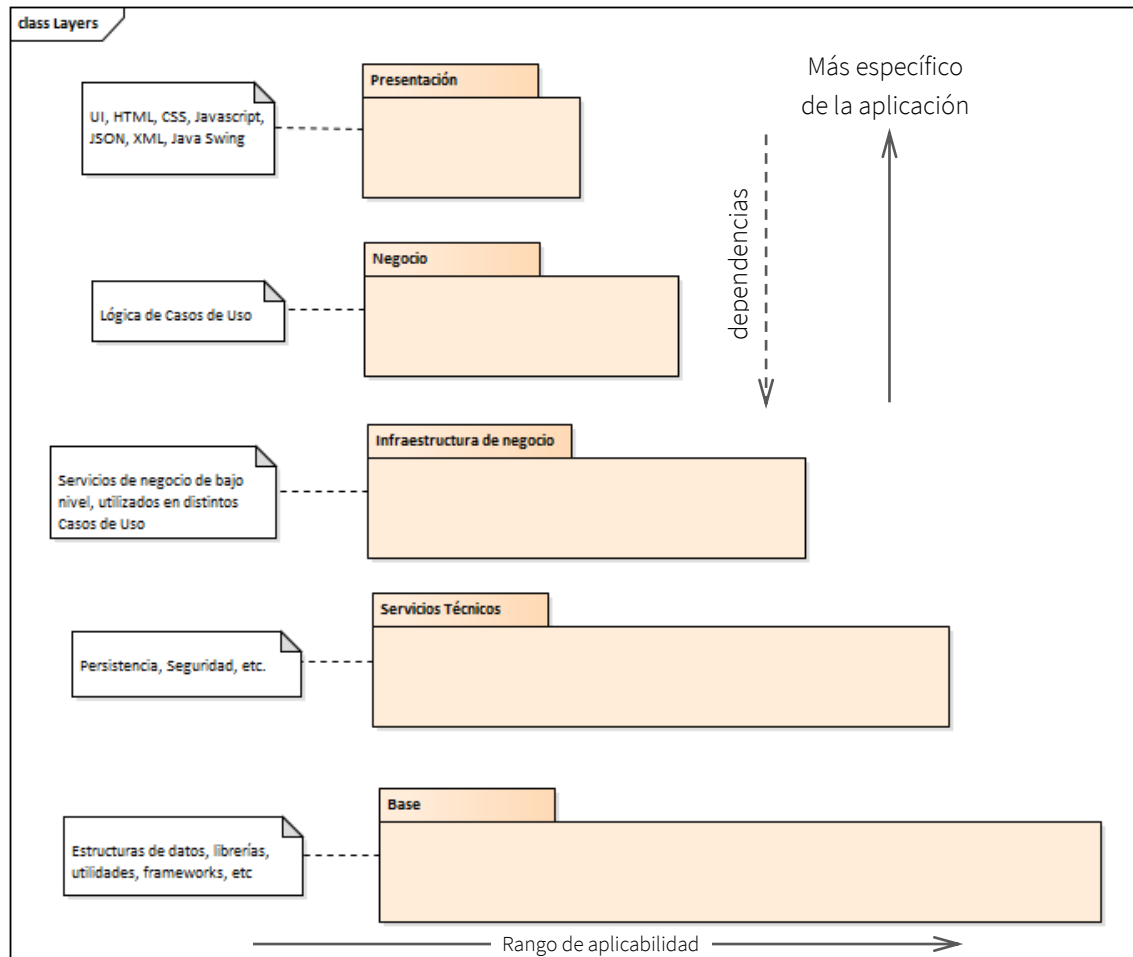
Problema	¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?
Solución	Asignar la responsabilidad de recibir o manejar un mensaje de evento del sistema a una clase que representa una de las siguientes opciones: • Representa el sistema global, dispositivo o subsistema (controlador de fachada). • Representa un escenario de caso de uso en el que tiene lugar el evento del sistema.

Un **evento del sistema** es un evento generado por un actor externo [1]. Se asocian con operaciones del sistema, tal como se relacionan los mensajes y los métodos. Puede ser presionar un botón, hacer *hover* en algún componente, completar un campo de un formulario, etc. Por ejemplo, en la pantalla de la derecha un evento del sistema es presionar el botón “Crear Caso”, que dispara la operación de la creación de un Caso:

Es muy común, a la hora de definir la arquitectura de un sistema, seguir una arquitectura **en capas**, basado en el patrón de arquitectura “Capas” (Layers). Una capa es una agrupación lógica de clases que tienen responsabilidades similares, diseñadas de forma cohesiva y con un bajo acoplamiento, de forma tal que se pueda modificar una capa sin afectar en gran medida al resto.

La imagen a continuación ilustra un ejemplo de aplicación de este patrón. Se ve que el sistema fue diseñado con 5 capas, cada una con una responsabilidad muy marcada y con dependencias únicamente hacia las capas inferiores (no puede haber conocimiento de las capas superiores). Este patrón resuelve

problemas de acoplamiento, cohesión, y reutilización a lo largo de todo el sistema, no solo para un caso particular.



Los eventos del sistema surgen en la capa de presentación, donde se encuentran los componentes visuales. Gestionar dichos eventos implica las siguientes responsabilidades:

- Coordinar la lógica de negocio que atiende el evento, ubicada en la capa de negocio.
- Validar que los tipos de datos ingresados por el actor del caso de uso sean correctos.
- Intercambiar información utilizando el formato y protocolo esperado por la capa de presentación. Distintas interfaces gráficas utilizan distintos formatos. Por mencionar algunos, las interfaces REST esperan que el formato sea JSON, SOAP espera que sea XML, un WebSocket maneja un mecanismo de comunicación completamente distinto, etc.

Asignar estas responsabilidades a la capa de negocio o a la capa de presentación es una solución que trae problemas de reutilización, cohesión, y acoplamiento:

- **Baja reutilización:** Si la capa de presentación se encarga de coordinar la lógica de negocio, un cambio en la misma implicaría modificar todas las interfaces gráficas que la utilizan. Además, los tests son más pesados, ya que para testear lógica de negocio hay que conocer detalles de presentación: Por ejemplo,

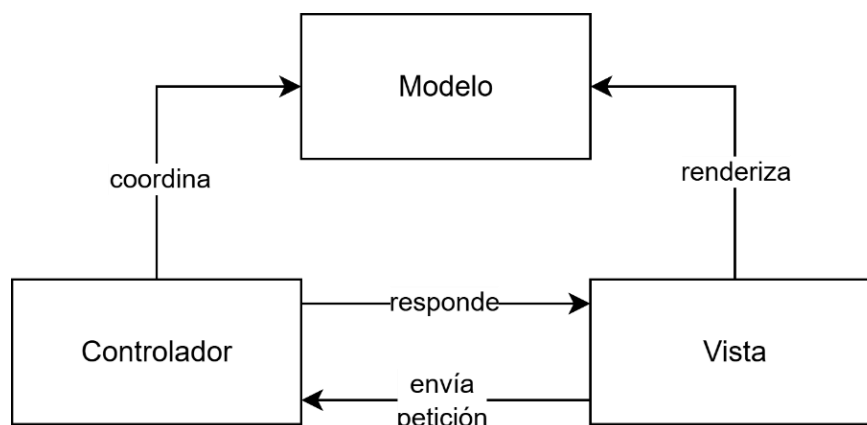
levantar un servidor web, cargar la interfaz gráfica, etc. Sería deseable poder reutilizarla de forma aislada independientemente de cómo vaya a ser consumida.

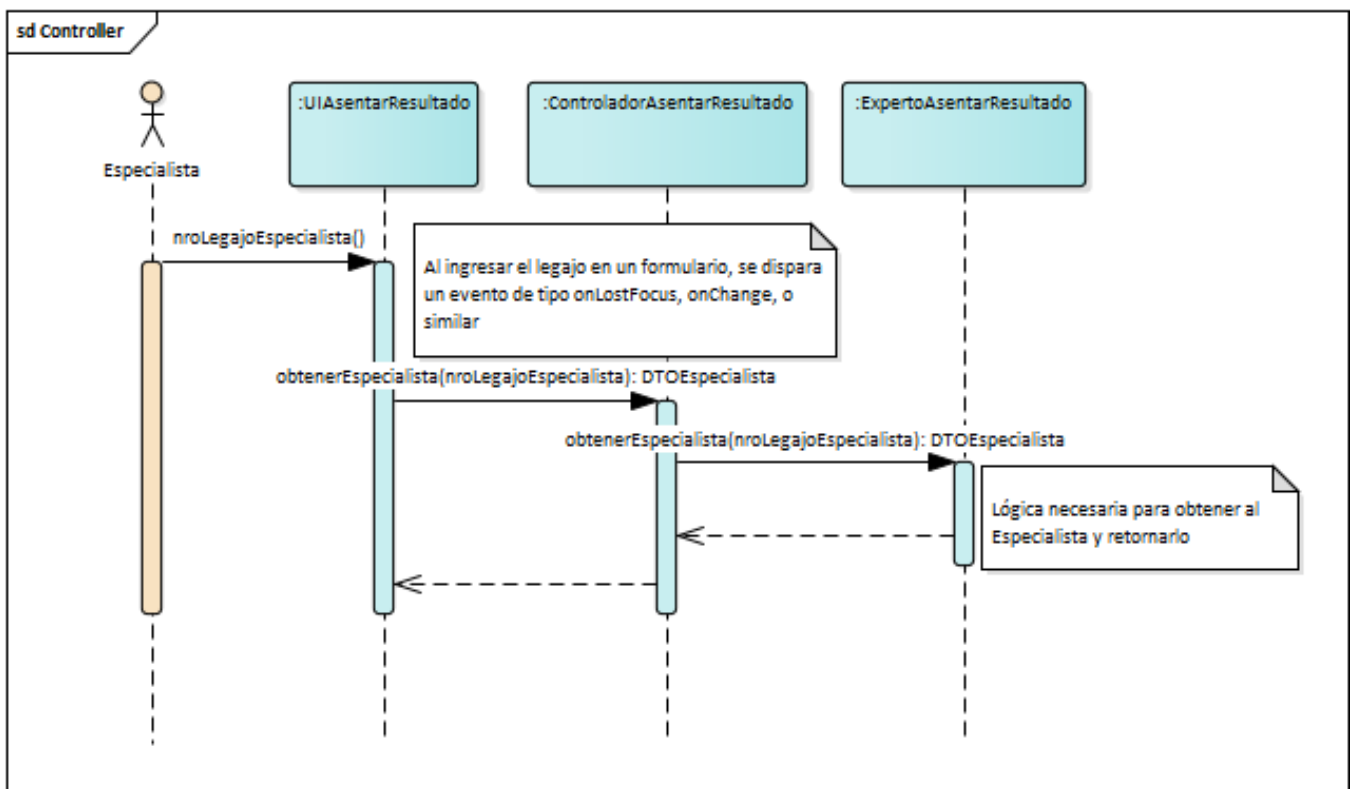
- Acoplamiento a los detalles de comunicación y formato de transferencia: La lógica de negocio queda acoplada al protocolo de comunicación y el formato para el intercambio de datos. Si se quisiese utilizar distintos tipos de interfaces gráficas, debería conocer todas las posibles formas de intercambiar información con cada una de ellas.
- No hay buena separación de responsabilidades: No es cohesivo que la capa de lógica de negocio conozca detalles técnicos que no tienen relación con los casos de uso. La forma de intercambiar información con la interfaz gráfica son aspectos no funcionales que no se relacionan con la lógica de negocio.

Un **controlador** es una clase encargada de atender eventos de entrada en el sistema. Al igual que la Indirección, es una aplicación del patrón Fabricación Pura. Si hay pocos eventos, se puede utilizar un único controlador (Controlador de Fachada), pero sino se utiliza un controlador por cada Caso de Uso. Por sí mismo, no realizan mucho trabajo. En su lugar delegan o coordinan actividades.

El patrón controlador ha evolucionado a lo largo del tiempo, variando el uso que se le da y las motivaciones que lo originan según el tipo de sistema. Sin embargo, la idea principal es atemporal: hay un conjunto de responsabilidades que no es un buen diseño asignarlas a la capa de presentación o a la capa de negocio, debido a la cohesión, acoplamiento, y reutilización. Entonces, se añade una nueva capa a la arquitectura, ubicada entre medio de la capa de presentación y la capa de lógica de negocio. Esta capa se suele llamar **capa de aplicación**, y es donde se encuentran los controladores, encargados de la comunicación entre las otras dos capas.

El patrón **MVC** aplica el patrón Controlador. No hay una única interpretación de este patrón, pero todas proponen que la *Vista* (capa de presentación) no debe interactuar directamente con el *Modelo* (capa de lógica de negocio, entidades), sino a través de un *Controlador*.





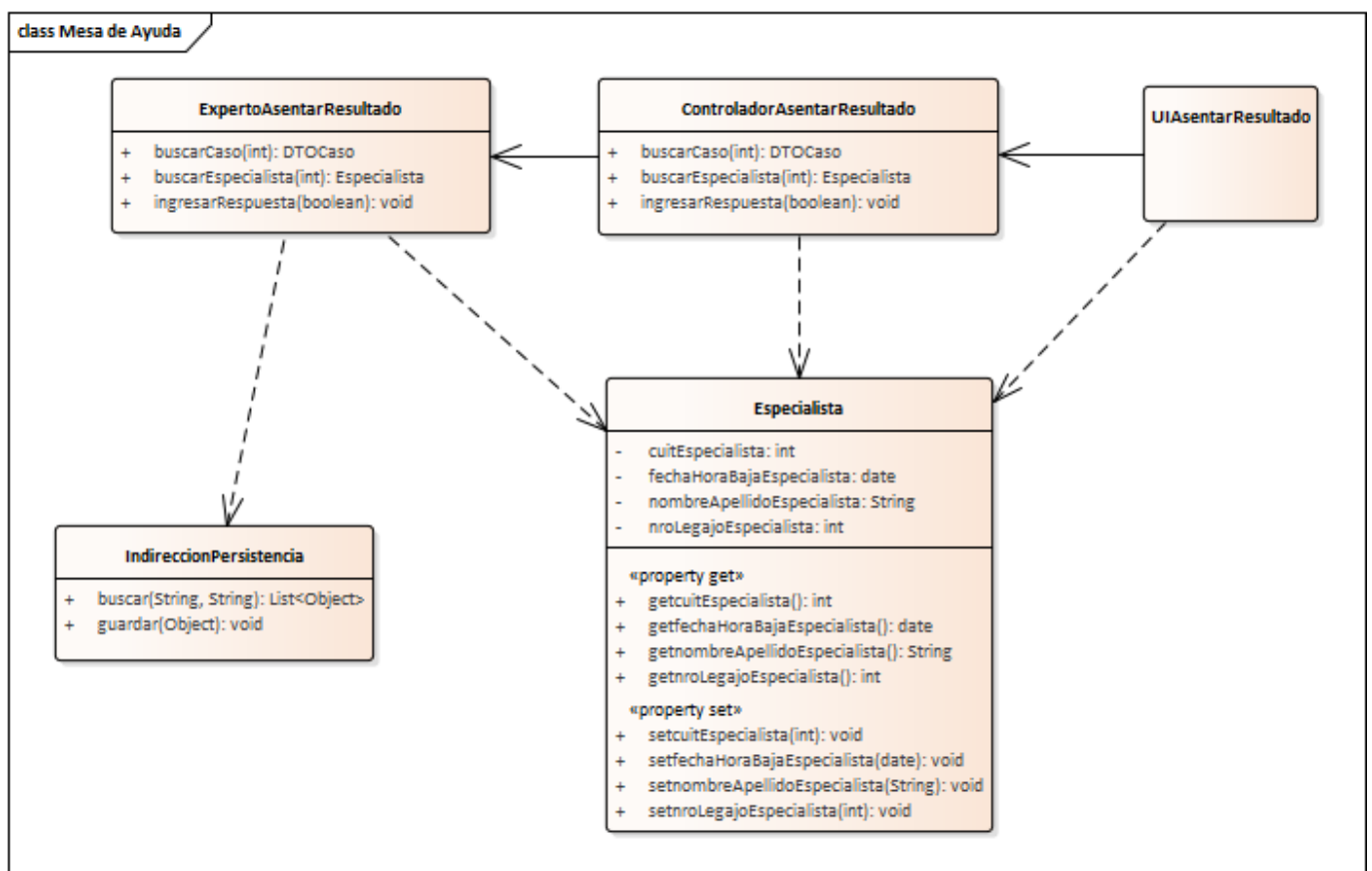
DTO

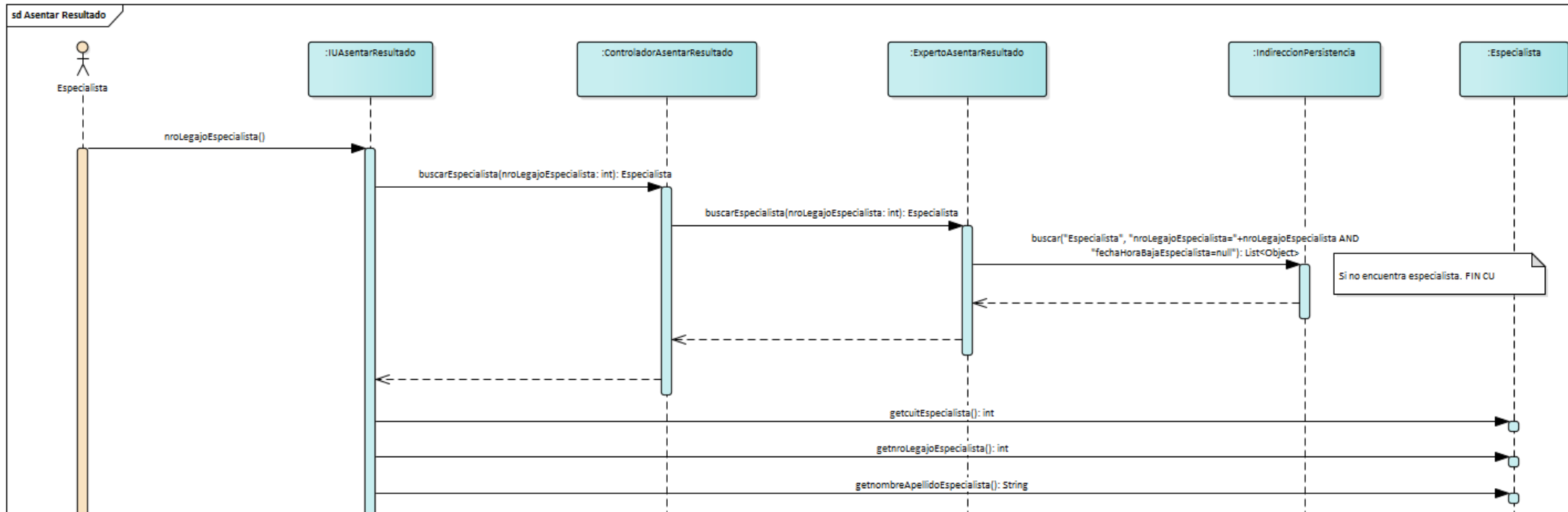
Problema	¿Cómo intercambiar datos entre diferentes capas de una aplicación de manera de mantener un bajo acoplamiento entre ellas?
Solución	Crear pseudo-entidades a medida de las necesidades. Es decir, definir clases con los atributos necesarios para representar los datos que se desean intercambiar entre las diferentes capas.

Es una situación común tener que intercambiar datos entre distintas capas, por ejemplo, las Entidades del sistema. Suponiendo que los especialistas del negocio de Mesa de Ayuda necesitan consultar sus propios datos ¿cómo se resolvería este problema?

Inicialmente, se puede pensar que se retorna a la interfaz la instancia de Especialista. Se muestra en el siguiente diagrama de secuencia que el especialista primero debe ingresar su legajo. La petición es recibida por el controlador quien delega en el Experto del Caso de Uso para luego buscar utilizando la indirección de persistencia en la base de datos al especialista. Finalmente, es retornado y se muestra por pantalla.

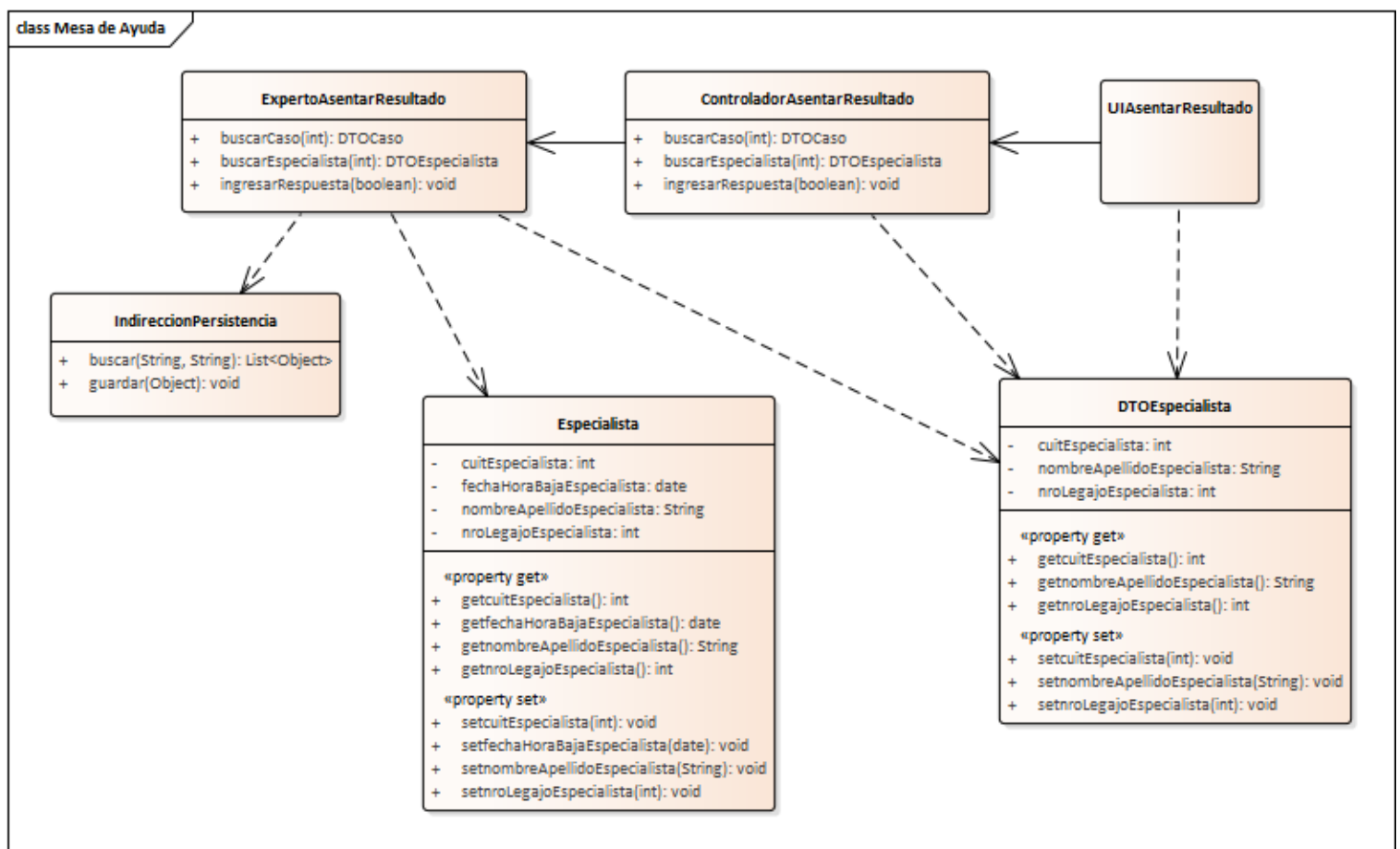
No obstante, la solución tiene problemas de acoplamiento. La capa de presentación tiene conocimientos de cómo se estructuran las entidades del sistema. Si ocurre un cambio en las mismas (por ejemplo, cambio en los atributos, relaciones, cardinalidades, navegabilidades, etc.) habrá que modificar cada interfaz gráfica que utilice a las entidades solicitadas. Es decir, la capa de presentación se encuentra fuertemente acoplada a la capa de negocio.

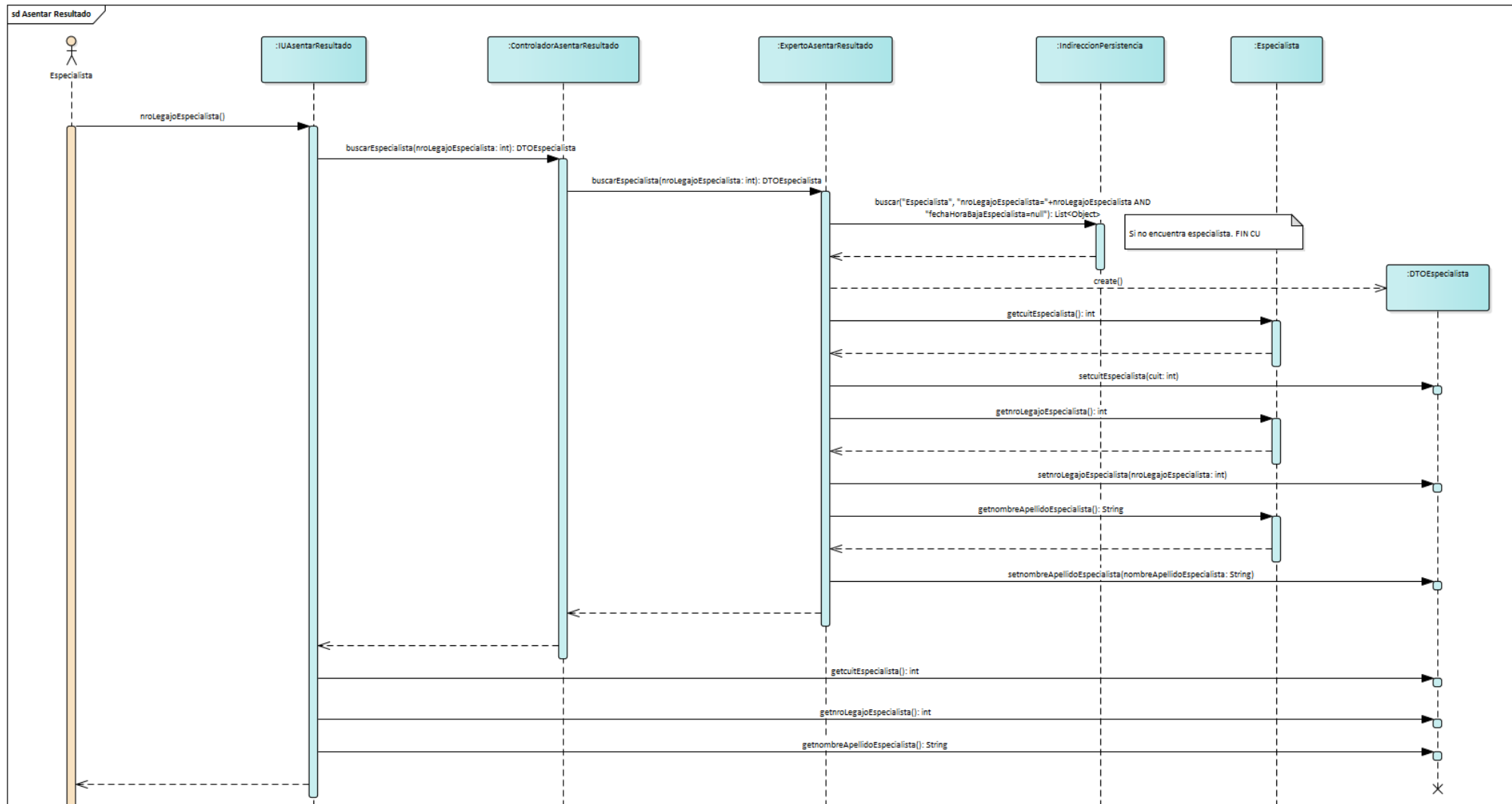




El patrón **DTO** (*Data Transfer Object*) propone resolver este problema definiendo **pseudoentidades** que contienen únicamente los datos que se requieren enviar. Son estructuras de datos diseñadas a medida de las necesidades, de forma conveniente para quienes requieran los datos. Por lo tanto, puede incluir solo los atributos y relaciones que sean de utilidad, independientemente de cómo se estructuren en la capa de negocio. Si hay un cambio en las entidades, únicamente hay que modificar el mapeo de la entidad al DTO, y no la lógica de los distintos consumidores de los datos.

Se muestra a continuación un diagrama de secuencia explicando el uso del patrón: cada vez que se requiera retornar un especialista, en lugar de retornarlo en sí mismo se creará una instancia del DTO, que sirve como “proyección” de lo que le interesa a la capa de presentación sobre la entidad.





Conclusión

Como ingenieros en sistemas, hay que actuar con profesionalismo. Parte de ese profesionalismo implica pensar en el futuro y en quienes tengan que hacer mantenimiento el día de mañana. El diseño de software no se centra únicamente en hacer sistemas que simplemente funcionan, a lo largo de este apunte se ha explorado cómo la asignación inteligente de responsabilidades determina la diferencia entre un sistema que cumple con los requisitos, y uno que además se adapta al cambio. La Alta Cohesión y el Bajo Acoplamiento funcionan como brújulas que orientan continuamente hacia soluciones mantenibles y escalables.

Los patrones GRASP, junto con otros patrones, representan principios fundamentales que todo diseñador debería interiorizar. Proporcionan guías de pensamiento que ayudan a tomar decisiones informadas en cada punto del diseño. Como tales, algunos tienen más de una forma de implementación.

La experiencia del diseñador juega un rol fundamental en la aplicación efectiva de estos patrones. Saber cuándo es apropiado protegerse de variaciones futuras y cuándo es mejor optar por la simplicidad requiere práctica, criterio, y un profundo entendimiento del dominio del problema.

Recordar estos principios y patrones permite no solo hacer sistemas que funcionan hoy, sino que sigan siendo útiles, comprensibles y modificables en el futuro. El verdadero valor de los patrones no está en memorizarlos, sino en interiorizarlos hasta que se conviertan en la forma natural de pensar sobre la estructura y responsabilidades de sus sistemas.

Preguntas de autoevaluación

Para evaluar el entendimiento del apunte, se plantean algunas preguntas para reflexionar sobre los temas abordados. Es recomendable intentar responderlas y luego debatirlas en clase o en consulta.

1. Se mencionó que el tipo de relación entre clases no es la única variable a la hora de considerar el acoplamiento. ¿Podrías plantear un ejemplo concreto?
2. En la materia, para la lógica de negocio del caso de uso, se utiliza una clase del estereotipo de análisis de control llamada *Experto<Nombre_del_Caso_de_Uso>*. ¿Te parece que dicha clase es una aplicación del patrón Experto en Información?
3. Se mencionó que se utiliza el patrón DTO para intercambiar datos entre capas del sistema. Además de utilizarlo para la capa de presentación y la de lógica de negocio ¿podría tener otra aplicación? Pensá en ejemplos.

4. ¿Podrías imaginar un ejemplo de los patrones Fabricación Pura o Indirección que no sean para la interacción con la base de datos? Definí la signatura de sus métodos, qué clases la utilizarían, y por qué sería útil usarla.
5. En el negocio de Mesa de Ayuda, se requiere mostrar al especialista los datos del Caso seleccionado, junto con el detalle de todos sus CasosInstancia (incluyendo su estado, tareas registradas, fechas de inicio y fin, y cualquier otro dato que consideres relevante). ¿Cómo aplicarías el patrón DTO en este caso?
6. ¿Qué relaciones encontrarías entre los patrones de este apunte y el artefacto “Descripción del Flujo de Sucesos de Caso de Uso”?

Bibliografía

- [1] Craig Larman (2008). GRASP: diseño de objetos con responsabilidades (cap. 16). GRASP: más patrones para asignar responsabilidades (cap. 22). Diseño de la arquitectura lógica con patrones (cap. 30). En *UML y Patrones* (2da edición). Prentice Hall.
- [2] Ivar Jacobson, Grady Booch, James Rumbaugh (2008). Artefacto: clase de análisis (cap. 8.4.2). En *El Proceso Unificado de Desarrollo de Software*. Pearson.
- [3] Roger S. Pressman (2010). Conceptos de Diseño (cap. 8). En *Ingeniería de Software: Un Enfoque Práctico* (7ma edición). Mc Graw Hill.